Transaction in raw Hibernate :

```
Session session =
        sessionFactory.openSession();

Transaction transaction =
        session.beginTransaction();

try {

    session.persist(student);

    transaction.commit();

} catch (Exception e) {

    transaction.rollback();

}
```

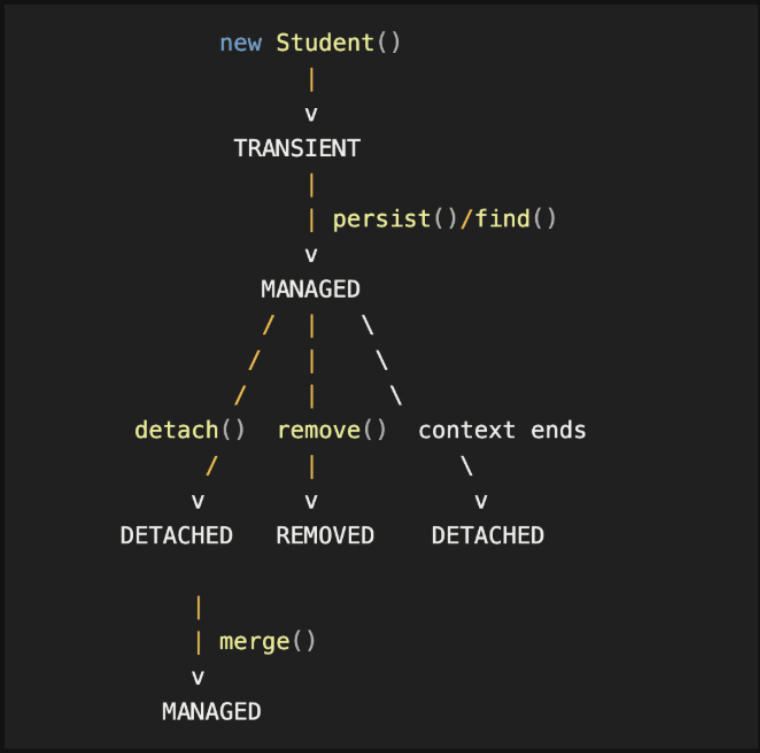
Transactions in Spring :

```
@Transactional

// example

@Transactional
public void createStudent() {

    // business logic

}
```

EntityManager lifecycle :

```
BEGIN

EntityManager OPEN
Persistence Context ACTIVE

    work...
    work...
    work...

FLUSH
COMMIT

EntityManager cleaned up
Persistence Context ends
```

Entity Lifecycle States Diagram :



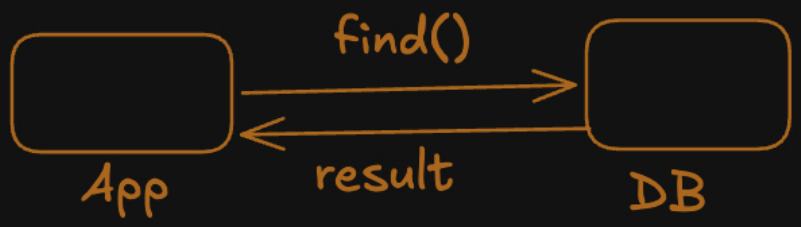
EntityManager
EntityManagerFactory
Persistence Context
Transactions

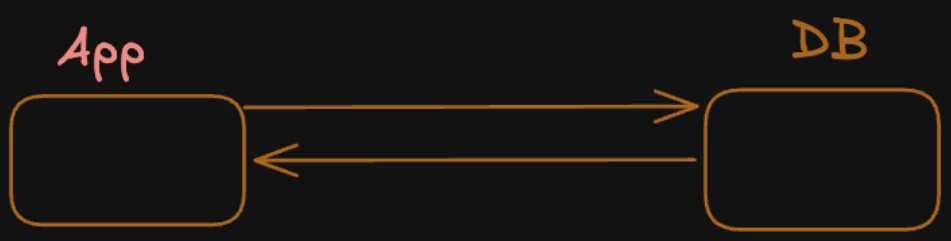
Hibernate --> find(), persist()

Student Id#1
Student Id#2

First Level Cache

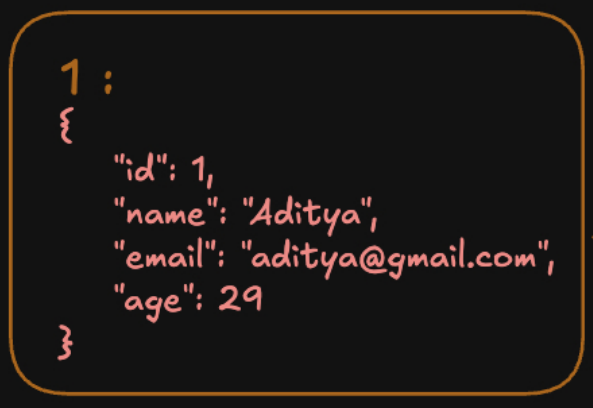
Persistence Context





SELECT * from students
WHERE id = 1

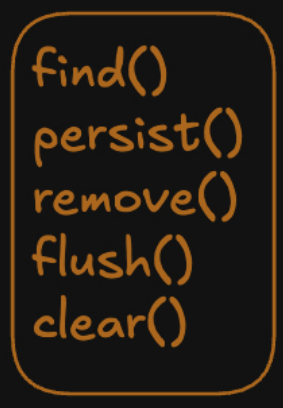
HEAP

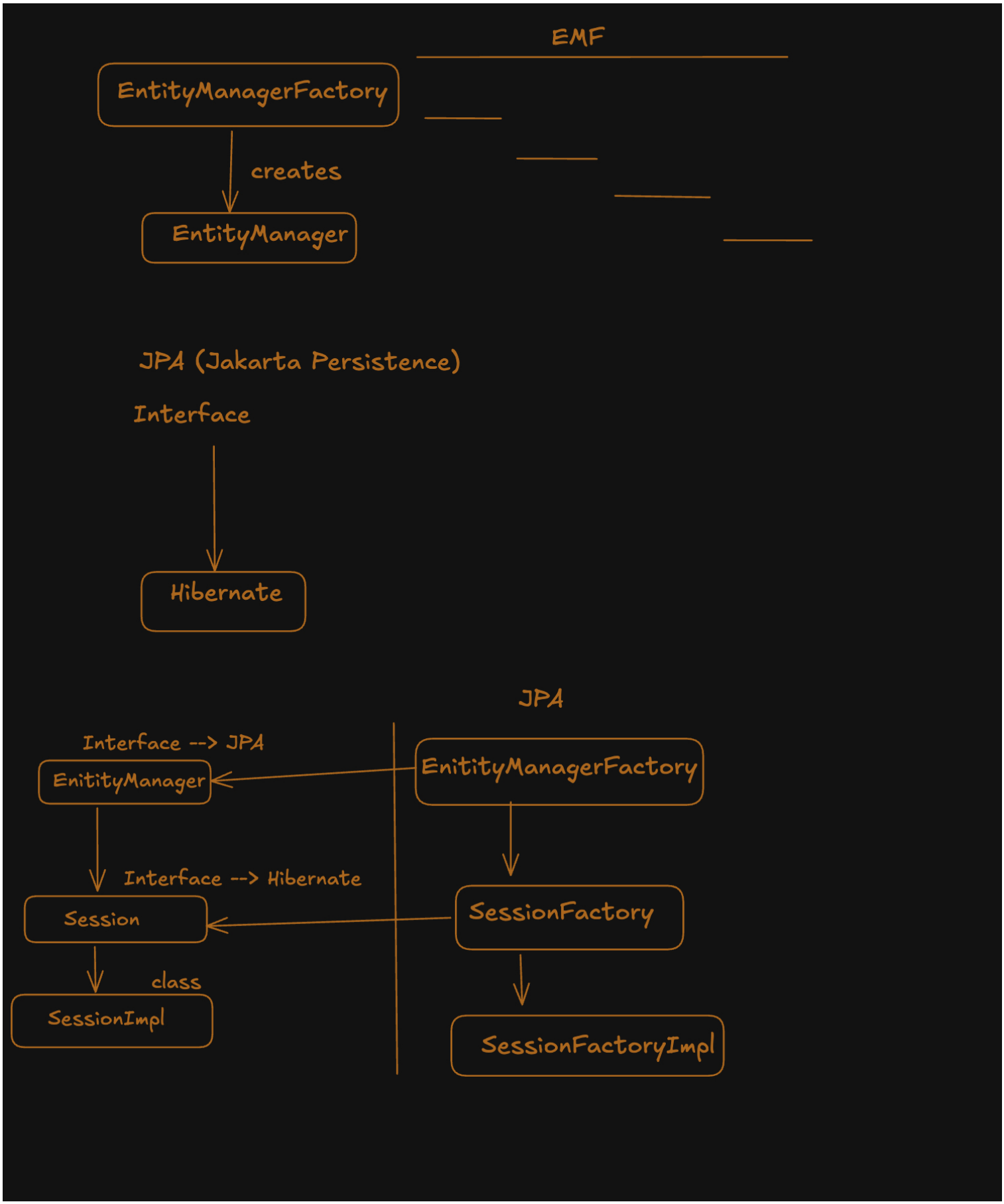


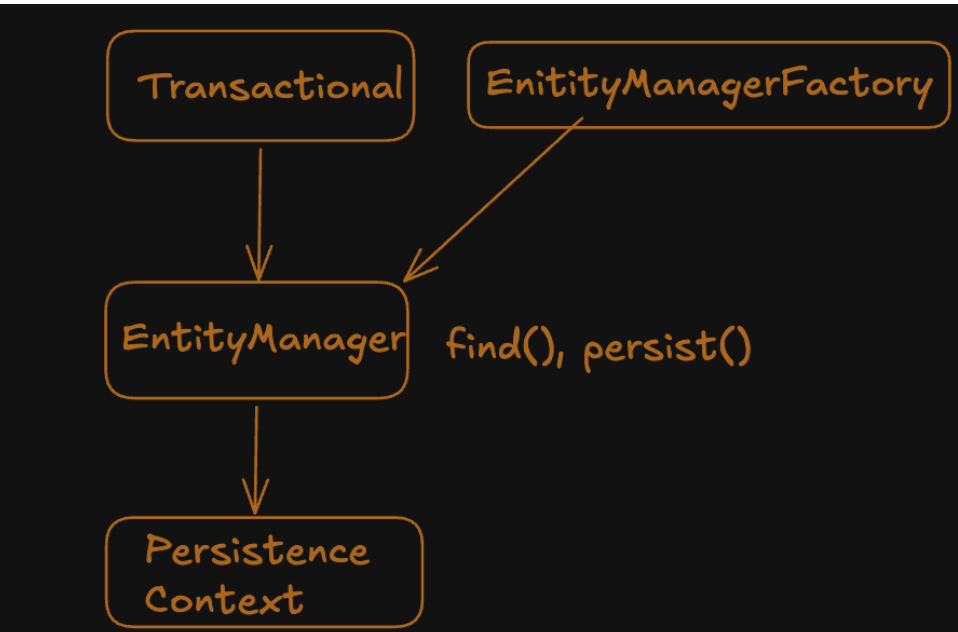
Persistence
Context

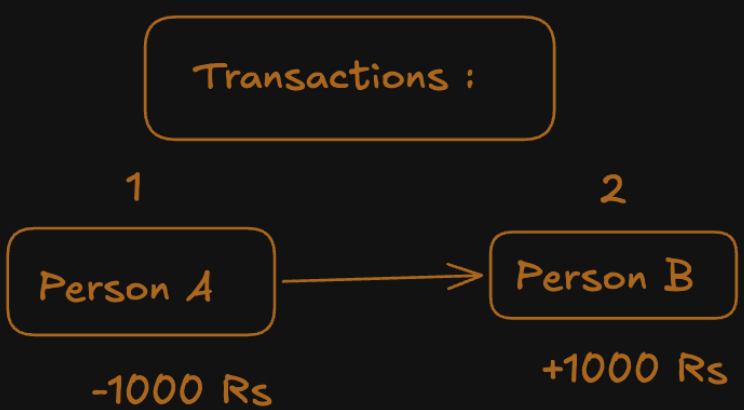
find(1)

UPDATE students
SET name = ROhit, age = 25



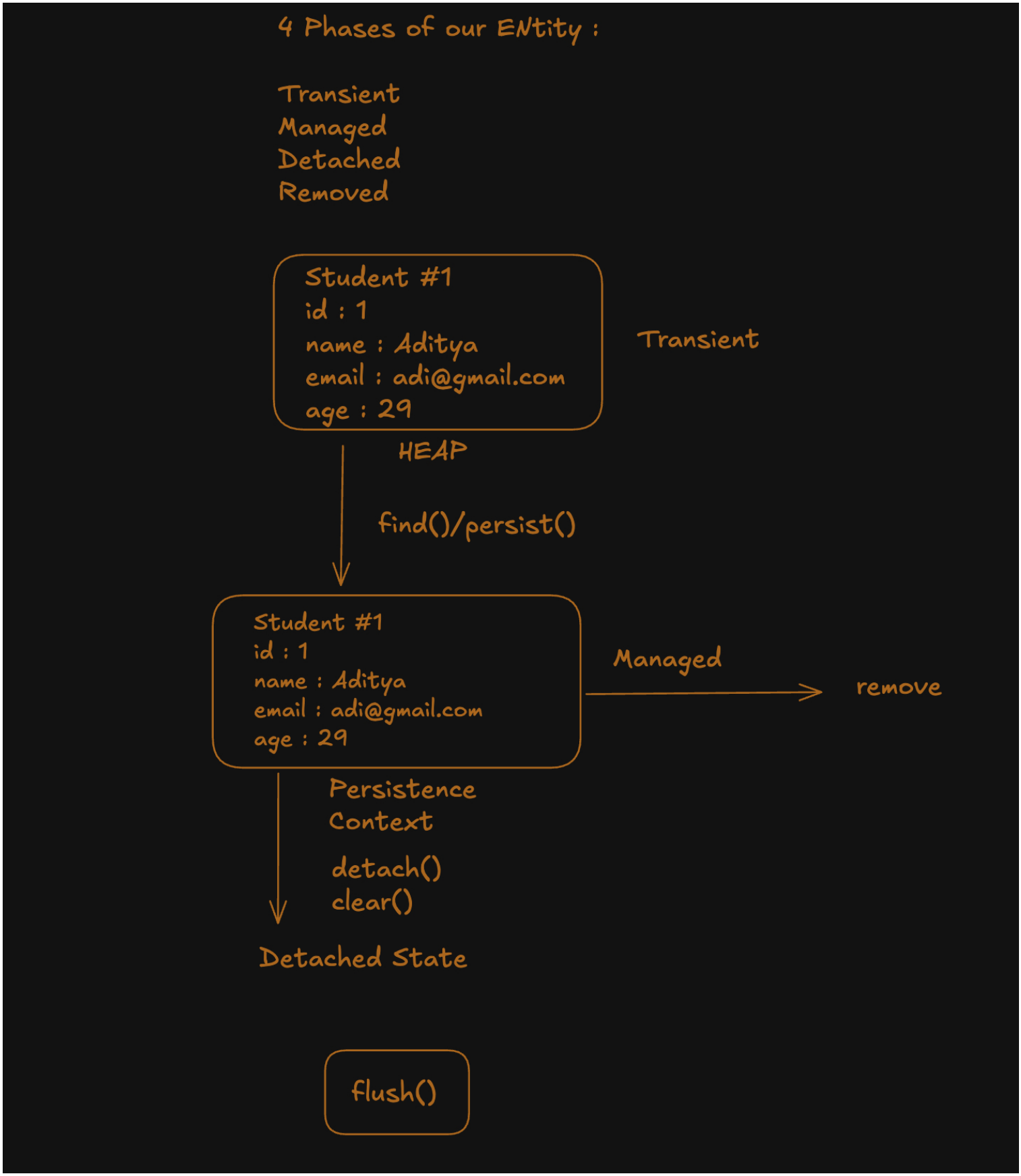

EMF
EntityManagerFactory
creates
EntityManager
JPA (Jakarta Persistence)
Interface
Hibernate
JPA
Interface --> JPA
EnitityManager
EnitityManagerFactory
Interface --> Hibernate
Session
SessionFactory
class
SessionImpl
SessionFactoryImpl






```
UPDATE accounts
SET balance = balance - 1000
WHERE id = 1

UPDATE accounts
SET balance = balance + 1000
WHERE id = 2
```

```
BEGIN

UPDATE A
UPDATE B ← Error

ROLL_BACK
```


4 Phases of our ENtity :
Transient
Managed
Detached
Removed
Student #1
id : 1
name : Aditya
email : adi@gmail.com
age : 29
Transient
HEAP
find()/persist()
Student #1
id : 1
name : Aditya
email : adi@gmail.com
age : 29
Managed
remove
Persistence
Context
detach()
clear()
Detached State
flush()